Classification: logistic regression and KNN

🎯 Today's goal

By the end of this hour you will:

- train a **logistic regression** and get **probabilities**, not just answers
- move the **decision threshold** and see what changes
- understand how **k-nearest neighbours** (KNN) votes, and how to choose `k`
- know why features must be put on the **same scale**, and do it with `StandardScaler`
- chain steps with a **pipeline** and compare models fairly on the same split

🤔 Why this matters

Is this e-mail spam? Will this customer leave? Is this transaction fraud? Most real machine-learning jobs are **classification**: putting each example into one of a few classes (groups). On Day 16 you trained a classifier and got answers. Today you'll also get the model's **confidence**, because "spam, 51% sure" and "spam, 99% sure" deserve different treatment.

Think of a weather forecast: it doesn't say "rain" or "no rain", it says "70% chance of rain", and *you* decide whether that's enough to take an umbrella. That decision point is the **threshold**.

🔬 The data: breast cancer measurements

scikit-learn includes a real, well-known dataset of 569 breast tissue samples. For each sample, a doctor took a microscope image of the cells, and a computer measured 30 features: the cells' size (radius, area), texture, smoothness, and so on. The label says whether the sample was **malignant** (cancer, `0`) or **benign** (not cancer, `1`).

> **Teaching example only:** we use this dataset because it's real, small and clean, which makes it perfect for practising. Nothing on this page is medical advice, and a model like this is *not* how anyone should be diagnosed.

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

cancer = load_breast_cancer(as_frame=True)
X = cancer.data
y = cancer.target

print("Samples and features:", X.shape)
print("Classes:", cancer.target_names.tolist())   # 0 malignant, 1 benign
print(y.value_counts())

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

```
Samples and features: (569, 30)
Classes: ['malignant', 'benign']
target
1    357
0    212
Name: count, dtype: int64
```

We split right away (Day 16), with `stratify=y` because there are more benign samples than malignant ones. Let's look at a few of the 30 features:

```
X[["mean radius", "mean texture", "mean area", "mean smoothness"]].describe().round(2)
```

	mean radius	mean texture	mean area	mean smoothness
count	569.00	569.00	569.00	569.00
mean	14.13	19.29	654.89	0.10
std	3.52	4.30	351.91	0.01
min	6.98	9.71	143.50	0.05
25%	11.70	16.17	420.30	0.09
50%	13.37	18.84	551.10	0.10
75%	15.78	21.80	782.70	0.11
max	28.11	39.28	2501.00	0.16

Keep an eye on the sizes of these numbers: the area goes up to 2501, the smoothness never gets above 0.17. That will matter later today.

📈 Logistic regression: a line that gives probabilities

Despite the word "regression" in its name, **logistic regression** is a *classification* model. It works in two steps:

1. Like yesterday's linear regression, it adds up the features times their slopes, which gives a score that can be any number.
2. It squeezes that score into a **probability** between 0 and 1 with an S-shaped curve: very negative scores become almost 0, very positive ones almost 1.

Let's start with just two features. The steps are the same as always: create → fit → predict → score:

```
from sklearn.linear_model import LogisticRegression

two = ["mean radius", "mean texture"]

logistic = LogisticRegression()
logistic.fit(X_train[two], y_train)

print(f"Accuracy with 2 features: {logistic.score(X_test[two], y_test):.1%}")
```

```
Accuracy with 2 features: 88.8%
```

🎲 `predict` versus `predict_proba`

`predict` gives the class. `predict_proba` ("proba" = probability) gives the chance of *each* class, one column per class in the order 0, 1:

```
first5 = X_test[two].head()

print("Classes:      ", logistic.predict(first5).tolist())
print("Real answers: ", y_test.head().tolist())
print()
print("Probabilities [malignant, benign]:")
print(logistic.predict_proba(first5).round(2))
```

```
Classes:       [1, 0, 0, 1, 0]
Real answers:  [1, 0, 1, 1, 0]

Probabilities [malignant, benign]:
[[0.07 0.93]
 [0.98 0.02]
 [0.55 0.45]
 [0.11 0.89]
 [0.65 0.35]]
```

Each row adds up to 1. The first sample is 93% benign: the model is quite sure. The third is 55% malignant: a coin flip, and in fact the model got it wrong. Probabilities tell you *which* answers to trust.

🎚️ The decision threshold

`predict` simply says "benign" when the chance of benign is at least **0.5**. That cut-off is the **threshold**, and you can choose a different one. Suppose a mistaken "benign" is much worse than a mistaken "malignant" (which would just lead to more tests). Then we might only say "benign" when the model is at least 90% sure:

```
chance_benign = logistic.predict_proba(X_test[two])[:, 1]   # column 1: benign

for threshold in [0.5, 0.9]:
    says_benign = chance_benign >= threshold
    # said benign, but was malignant:
    wrong_benign = (says_benign & (y_test == 0)).sum()
    print(f"Threshold {threshold}: {says_benign.sum()} called benign, "
          f"of which really malignant: {wrong_benign}")
```

```
Threshold 0.5: 92 called benign, of which really malignant: 9
Threshold 0.9: 57 called benign, of which really malignant: 1
```

`[:, 1]` means "all rows, column 1" (Day 9), and `&` means "and" for arrays (Day 11). A higher threshold makes fewer "benign" calls, but those it makes are much safer. Choosing the threshold

isn't a maths question; it depends on what each kind of mistake costs. You'll measure this properly on Day 20.

🏘️ K-nearest neighbours: ask the neighbours

On Day 16 you used KNN without looking inside. Its idea: to classify a new sample, find the `k` training samples that are **closest** to it (most similar measurements) and let them **vote**. There's no formula to learn; "fitting" just means remembering the training data.

Here's one made-up new sample and its 5 nearest neighbours:

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train[two], y_train)

# a made-up new sample
new_sample = pd.DataFrame({"mean radius": [13.0], "mean texture": [20.0]})
distances, positions = knn.kneighbors(new_sample)

print("Distances to the 5 neighbours:", distances[0].round(2).tolist())
print("Their labels:", y_train.iloc[positions[0]].tolist())
print("Vote result:", knn.predict(new_sample)[0])
print("Votes [malignant, benign]:", knn.predict_proba(new_sample)[0].tolist())
```

```
Distances to the 5 neighbours: [0.27, 0.4, 0.48, 0.55, 0.56]
Their labels: [1, 0, 1, 1, 1]
Vote result: 1
Votes [malignant, benign]: [0.2, 0.8]
```

Four neighbours say benign (1), one says malignant (0), so the answer is benign, and KNN's "probability" is simply the share of votes: 4 out of 5 = 0.8.

Choosing `k`: with `k=1` a single odd neighbour decides everything (too nervous). With a huge `k` the vote includes samples that aren't similar at all (too lazy). And `k` can't be bigger than the number of training samples:

💡 This example has a mistake on purpose. Run it and read the error.

```
too_many = KNeighborsClassifier(n_neighbors=500)
too_many.fit(X_train, y_train)          # only 426 training samples
too_many.predict(X_test)
```

⚠️ ValueError

The kind of value is right but the value itself isn't, like int("hello").

```
Traceback (most recent call last):
  line 3, in <module>
    too_many.predict(X_test)
    ~~~~~~~~~~~~~~~~^^^^^^^^
ValueError: Expected n_neighbors <= n_samples_fit, but n_neighbors = 500,
n_samples_fit = 426, n_samples = 143
```

Read the last line: *Expected n_neighbors <= n_samples_fit*. You can't ask 500 neighbours when only 426 exist.

⚖️ Why features must be on the same scale

"Closest" is measured as a distance, by adding up the differences in every feature. Now remember the sizes: a difference in *area* can be hundreds, a difference in *smoothness* is about 0.01. So KNN on all 30 raw features basically only looks at area and ignores the rest. It's like comparing people by their salary in cents and their height in kilometres.

The fix is **scaling**: rewrite each feature so that it has mean 0 and standard deviation 1 (Day 9: `std` measures the typical spread). Then a "big difference" means the same for every feature. `StandardScaler` does it:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train)    # learn each column's mean and std (training set!)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)    # same numbers on the test set

print("Area before: ", X_train["mean area"].head(3).round(1).tolist())
print("Area after:  ", X_train_scaled[:3, 3].round(2).tolist())
print(f"Mean now: {X_train_scaled.mean():.1f}, std now: {X_train_scaled.std():.1f}")
```

```
Area before:  [1214.0, 515.9, 912.7]
Area after:   [1.63, -0.39, 0.76]
Mean now: 0.0, std now: 1.0
```

Fit the scaler on the training set only. The test set plays the role of future data you haven't seen yet. If its numbers helped decide the mean and std, a little of the "exam" would leak into the "practice", and your score would be too optimistic.

Does it help? Compare KNN on raw and on scaled features:

```
raw_knn = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)
scaled_knn = KNeighborsClassifier(n_neighbors=5).fit(X_train_scaled, y_train)

print(f"KNN on raw features:    {raw_knn.score(X_test, y_test):.1%}")
print(f"KNN on scaled features: {scaled_knn.score(X_test_scaled, y_test):.1%}")
```

```
KNN on raw features:    93.0%
KNN on scaled features: 97.9%
```

🔗 Pipelines: scaling and model in one

Remembering to scale the training set, then the test set, then every new sample, is easy to get wrong. A **pipeline** glues the steps together into one model. When you call `fit`, it fits the scaler and then the model on the training data; when you call `predict` or `score`, it scales the new data first, automatically:

```
from sklearn.pipeline import make_pipeline

knn_pipe = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
knn_pipe.fit(X_train, y_train)     # raw data in, scaling happens inside
print(f"KNN pipeline: {knn_pipe.score(X_test, y_test):.1%}")
```

```
KNN pipeline: 97.9%
```

Logistic regression likes scaled features too. Without scaling, it has trouble finding its best slopes on all 30 features and stops after its limit of tries (`max_iter`, 100 by default) with a warning. Run this box and read the yellow message:

```
unscaled = LogisticRegression()
unscaled.fit(X_train, y_train)
print(f"Logistic regression, raw features: {unscaled.score(X_test, y_test):.1%}")
```

```
/lib/python3.14/site-packages/sklearn/linear_model/_logistic.py:406:
ConvergenceWarning: lbfgs failed to converge after 100 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-
regression
  n_iter_i = _check_optimize_result(
Logistic regression, raw features: 96.5%
```

"Failed to converge" means "I ran out of tries before I was done". You could allow more tries with `LogisticRegression(max_iter=5000)`, but that's slow. Scaling fixes the cause:

```
logistic_pipe = make_pipeline(StandardScaler(), LogisticRegression())
logistic_pipe.fit(X_train, y_train)
print(f"Logistic regression pipeline: {logistic_pipe.score(X_test, y_test):.1%}")
```

```
Logistic regression pipeline: 98.6%
```

🔢 Trying several values of `k`

A loop (Day 5) tries several `k` values and keeps every score in a dictionary (Day 3):

```
k_scores = {}
for k in [1, 3, 5, 7, 9, 15, 25]:
    pipe = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    pipe.fit(X_train, y_train)
    k_scores[k] = pipe.score(X_test, y_test)
    print(f"k = {k:2}: {k_scores[k]:.1%}")

best_k = max(k_scores, key=k_scores.get)
print("Best k:", best_k)
```

```
k =  1: 95.1%
k =  3: 97.2%
k =  5: 97.9%
k =  7: 97.9%
k =  9: 96.5%
k = 15: 95.8%
k = 25: 94.4%
Best k: 5
```

`max(k_scores, key=k_scores.get)` finds the key whose value is biggest (Day 6's `key=` trick). If several `k` tie, it returns the first one. The middle values work best: `k=1` is too nervous, `k=25` too lazy.

Honest note: choosing `k` by looking at the test score lets the test set influence the model a little. On Day 20 you'll learn cross-validation, the proper way to choose settings.

🏁 Comparing models fairly

A fair comparison uses the **same** training and test sets for every model, plus a baseline (Day 16). A dictionary of models and a loop keep it tidy:

```
from sklearn.dummy import DummyClassifier

models = {
    "baseline": DummyClassifier(strategy="most_frequent"),
    "logistic": make_pipeline(StandardScaler(), LogisticRegression()),
    "knn": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=best_k)),
}

results = {}
for name, m in models.items():
    m.fit(X_train, y_train)
    results[name] = m.score(X_test, y_test)

pd.Series(results).sort_values(ascending=False).round(3)
```

```
logistic    0.986
knn         0.979
baseline    0.629
dtype: float64
```

Both real models beat the baseline (which always says "benign") by a wide margin, and with scaling they are close to each other. When two models score about the same, people often pick the simpler one, or the one whose probabilities are more useful.

⚠️ Common mistakes

- Thinking logistic *regression* predicts numbers. It's a classifier that gives probabilities.
- Forgetting that `predict_proba` has one column per class: `[:, 1]` is the chance of class 1.
- Using KNN (or logistic regression) on features with very different scales. Scale first.
- Fitting the scaler on all the data, or on the test set. Fit it on the training set only (a pipeline does this for you).
- Choosing `n_neighbors` bigger than the training set → `ValueError`.
- Comparing models that were tested on different splits.

✅ Recap

- `LogisticRegression`: a weighted sum squeezed into a probability. `predict_proba` gives the chances, `predict` uses a 0.5 threshold.
- Your own threshold: `model.predict_proba(X)[:, 1] >= 0.9`.
- KNN: the `k` closest training samples vote. Small `k` = nervous, big `k` = lazy.
- `StandardScaler` gives every feature mean 0 and std 1. Fit on the training set only.
- `make_pipeline(StandardScaler(), model)` scales and models in one step.
- Compare models on the same split, with a baseline.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. A first logistic regression

Train a `LogisticRegression` called `model` on the two features in `two` (training set only), and store its accuracy on the test set in `accuracy`.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

cancer = load_breast_cancer(as_frame=True)
X, y = cancer.data, cancer.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
two = ["mean radius", "mean texture"]

# model = ...
# accuracy = ...
```

Hints:

1. Only the two columns: `X_train[two]` and `X_test[two]`.
2. `model = LogisticRegression()`, then `model.fit(X_train[two], y_train)`.
3. `accuracy = model.score(X_test[two], y_test)`

2. How sure is the model?

The starter trains a scaled logistic regression on all 30 features. Store the chance of **benign** (class 1) for every test sample in `chance_benign`. Then store in `sure_count` how many test samples have a chance of benign of **at least 0.9**.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

cancer = load_breast_cancer(as_frame=True)
X, y = cancer.data, cancer.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
model = make_pipeline(StandardScaler(), LogisticRegression())
model.fit(X_train, y_train)

# chance_benign = ...
# sure_count = ...
```

Hints:

1. `model.predict_proba(X_test)` has two columns: [malignant, benign].
2. Column 1 of every row: `[:, 1]`.
3. `(chance_benign >= 0.9)` gives True/False; `.sum()` counts the Trues.

3. Scaled KNN

Build a pipeline called `model` that first scales the features with `StandardScaler` and then uses `KNeighborsClassifier(n_neighbors=5)`. Fit it on the training set and store its test accuracy in `accuracy`.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

cancer = load_breast_cancer(as_frame=True)
X, y = cancer.data, cancer.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

# model = make_pipeline(...)
```

Hints:

1. The steps go inside `make_pipeline( , )` in order: scaler first, then the model.
2. `model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))`
3. Then `model.fit(X_train, y_train)` and `accuracy = model.score(X_test, y_test)`.

4. Find the best k

For each `k` in `[1, 3, 5, 7, 9, 11]`, fit a scaled KNN pipeline and store its test accuracy in a dictionary called `scores` (key = k, value = accuracy). Then store the `k` with the highest accuracy in `best_k`.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

cancer = load_breast_cancer(as_frame=True)
X, y = cancer.data, cancer.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

scores = {}
# loop over the k values here

# best_k = ...
```

Hints:

1. `for k in [1, 3, 5, 7, 9, 11]:` and inside it build a new pipeline with `KNeighborsClassifier(n_neighbors=k)`.
2. Inside the loop: fit it, then `scores[k] = pipe.score(X_test, y_test)`.
3. `best_k = max(scores, key=scores.get)`

5. Challenge: compare three models

Fit the three models in `models` on the same training set. Store each test accuracy in a dictionary called `results` with the same keys (`"baseline"`, `"logistic"`, `"knn"`). Then store the name of the best model in `best_name` and print a line like `Best: logistic (98.8%)`.

Your code should still work if someone adds a fourth model to the dictionary.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

cancer = load_breast_cancer(as_frame=True)
X, y = cancer.data, cancer.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

models = {
    "baseline": DummyClassifier(strategy="most_frequent"),
    "logistic": make_pipeline(StandardScaler(), LogisticRegression()),
    "knn": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=7)),
}

# results = ...
# best_name = ...
```

Hints:

1. Start with `results = {}` and loop with `for name, model in models.items():`
2. Inside the loop: `model.fit(X_train, y_train)` and `results[name] = model.score(X_test, y_test)`.
3. `best_name = max(results, key=results.get)`, then `print(f"Best: {best_name} ({results[best_name]:.1%})")`

🔜 Tomorrow

Decision trees and random forests. A decision tree classifies like a game of 20 questions ("Is the alcohol above 13? Then is the colour..."), and you can read its rules in plain text. You'll see how deep trees memorise, and how a whole "forest" of trees voting together gives steadier answers, using a dataset of wines.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. A first logistic regression

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

cancer = load_breast_cancer(as_frame=True)
X, y = cancer.data, cancer.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
two = ["mean radius", "mean texture"]

model = LogisticRegression()
model.fit(X_train[two], y_train)
accuracy = model.score(X_test[two], y_test)
print(f"Accuracy: {accuracy:.1%}")
```

2. How sure is the model?

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

cancer = load_breast_cancer(as_frame=True)
X, y = cancer.data, cancer.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
model = make_pipeline(StandardScaler(), LogisticRegression())
model.fit(X_train, y_train)

chance_benign = model.predict_proba(X_test)[:, 1]
sure_count = (chance_benign >= 0.9).sum()
print(sure_count, "of", len(X_test), "samples are at least 90% benign")
```

3. Scaled KNN

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

cancer = load_breast_cancer(as_frame=True)
X, y = cancer.data, cancer.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)
print(f"Accuracy: {accuracy:.1%}")
```

4. Find the best k

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

cancer = load_breast_cancer(as_frame=True)
X, y = cancer.data, cancer.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

scores = {}
for k in [1, 3, 5, 7, 9, 11]:
    pipe = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
    pipe.fit(X_train, y_train)
    scores[k] = pipe.score(X_test, y_test)

best_k = max(scores, key=scores.get)
print(scores)
print("Best k:", best_k)
```

5. Challenge: compare three models

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

cancer = load_breast_cancer(as_frame=True)
X, y = cancer.data, cancer.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

models = {
    "baseline": DummyClassifier(strategy="most_frequent"),
    "logistic": make_pipeline(StandardScaler(), LogisticRegression()),
    "knn": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=7)),
}

results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    results[name] = model.score(X_test, y_test)

best_name = max(results, key=results.get)
print(f"Best: {best_name} ({results[best_name]:.1%})")
```